

AWS Image Resizing and Rekognition System

Services Used

Amazon S3, AWS Lambda, Amazon Rekognition, Amazon SNS, AWS IAM, CloudWatch Logs, Pillow Python Library

Description

This project implements a serverless AI-based image processing pipeline using AWS cloud services. The system automatically processes images uploaded to Amazon S3, resizes images using Pillow inside AWS Lambda, detects labels using Amazon Rekognition, stores metadata output in S3, and sends notifications using Amazon SNS.

The architecture follows an event-driven serverless approach where AWS services automatically interact with each other without managing servers manually.

Definition

A serverless image processing pipeline is a cloud-based architecture where uploaded images are automatically processed without manually managing servers. AWS services trigger events automatically to resize images, analyze image content using AI, store output files, and send notifications.

Scope of the Project

- Build a serverless image processing workflow using AWS
- Automatically process uploaded images
- Resize images using AWS Lambda and Pillow
- Detect labels using Amazon Rekognition AI
- Store resized images and JSON metadata in S3
- Send notifications using Amazon SNS
- Implement event-driven architecture
- Learn serverless cloud automation

This Project Covers

- S3 bucket configuration
- Lambda function creation
- IAM role and permissions
- Rekognition AI integration
- SNS notification workflow
- Pillow image resizing
- JSON metadata generation
- CloudWatch monitoring
- Event-driven architecture
- Serverless application workflow

Methodology

1. Serverless Event-Driven Architecture (Implemented Method)

- Uses S3, Lambda, Rekognition, SNS, and IAM
- Images uploaded to S3 automatically trigger Lambda
- Lambda resizes image and detects labels using AI

- Processed output is stored in S3 automatically
- SNS sends email notifications after processing
- No server management required

2. Traditional Server-Based Processing (Alternative Method)

- Images processed manually on EC2 servers
- Requires server management and scaling
- Higher operational overhead
- Not used in this project because serverless architecture is more scalable and cost efficient

Project Overview

This project implements a serverless image processing pipeline using AWS cloud services. The system automatically processes images uploaded to Amazon S3, resizes them using AWS Lambda and Pillow, detects image labels using Amazon Rekognition AI, stores the processed output in S3, and sends notifications using Amazon SNS.

The architecture follows an event-driven serverless approach where AWS services automatically interact with each other without managing servers manually.

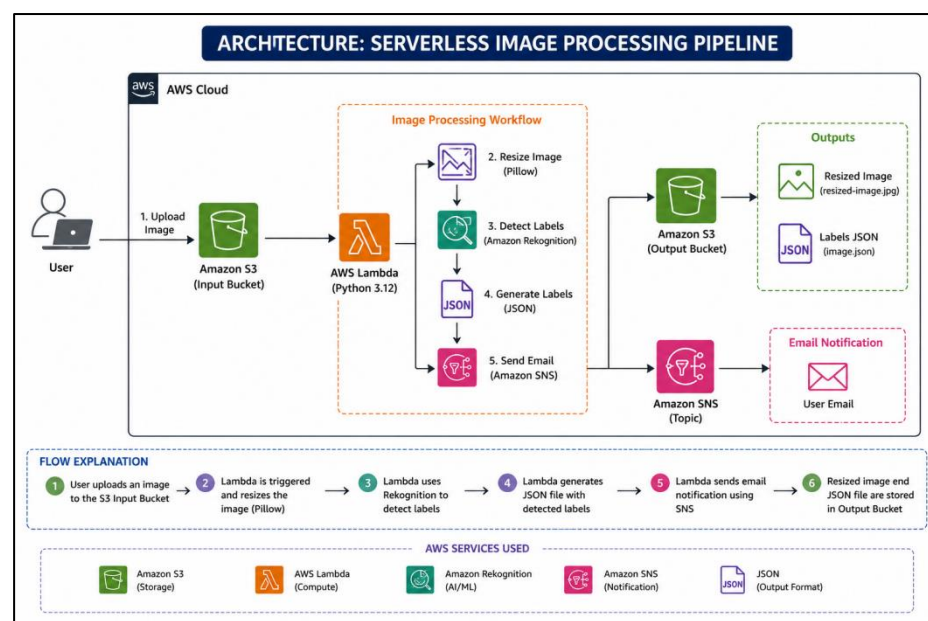
Project Objective

The objective of this project is to build a scalable and automated image processing system using AWS serverless services.

The application should:

- Automatically detect new image uploads
- Resize uploaded images
- Detect labels and objects inside images using AI
- Store resized images and JSON metadata in S3
- Send email notifications after processing

Architecture Diagram



AWS Services Explanation

1. Amazon S3

Definition

Amazon S3 is an object storage service used to store files, images, videos, and application data.

Purpose in Project

- Stores uploaded images
- Stores resized images
- Stores JSON metadata files

2. AWS Lambda

Definition

AWS Lambda is a serverless compute service that automatically runs code when triggered.

Purpose in Project

- Triggered automatically when image is uploaded to S3
- Resizes image using Pillow
- Calls Rekognition API
- Saves JSON output to S3
- Sends SNS email notification

3. Amazon Rekognition

Definition

Amazon Rekognition is an AWS AI service used for image and video analysis.

Purpose in Project

Rekognition analyzes uploaded images and detects:

- Faces
- Objects
- Labels
- People
- Emotions
- Animals

4. Amazon SNS

Definition

Amazon SNS is a messaging and notification service.

Purpose in Project

- Sends email notifications after image processing

5. AWS IAM

Definition

IAM controls permissions and access between AWS services.

Purpose in Project

The Lambda execution role was configured with permissions for:

- AmazonS3FullAccess
- AmazonRekognitionFullAccess
- AmazonSNSFullAccess
- AWSLambdaBasicExecutionRole

Implementation Steps:

Step 1: Create S3 Buckets

Two S3 buckets were created:

Input Bucket

Used to upload original images.

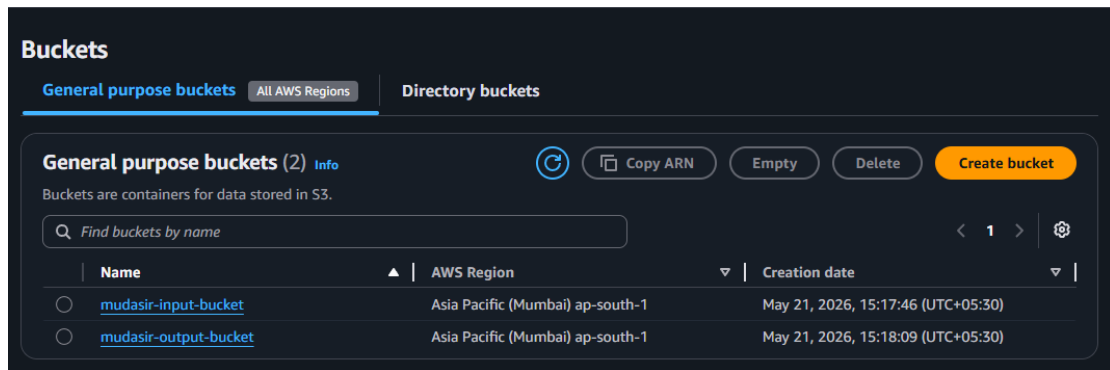
Output Bucket

Used to store:

- Resized images
- JSON metadata files

Bucket Settings:

- Block all public access enabled
- Same AWS region used for all services

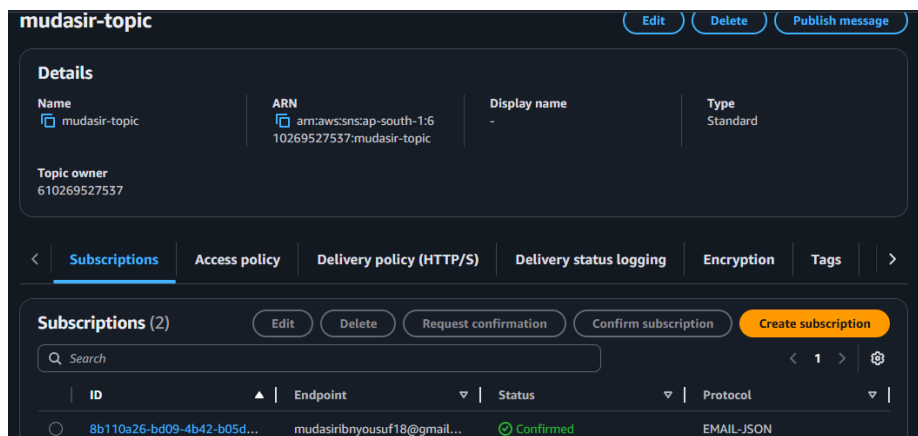


Step 2: Create SNS Topic

An SNS topic was created for email notifications.

Configuration

- Type: Standard
- Email subscription added
- Subscription confirmed through email



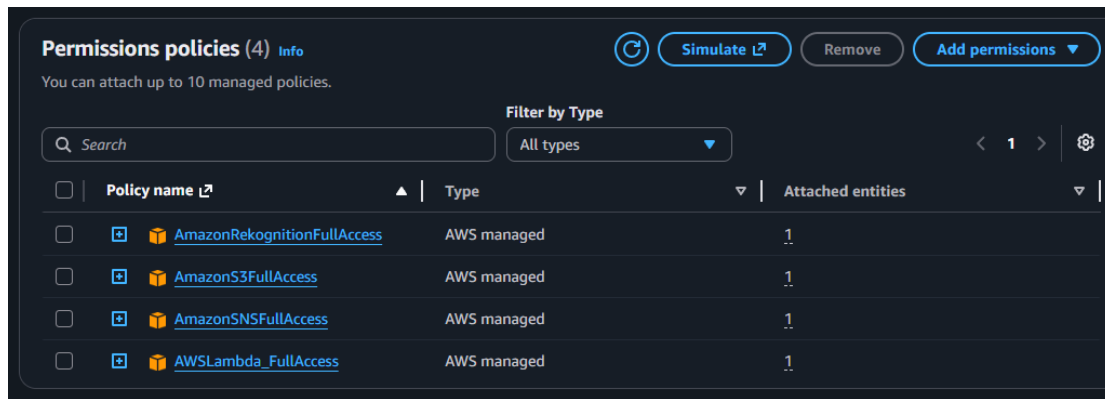
Step 3: Create IAM Role

An IAM role was created for Lambda execution.

Policies Attached

- AmazonS3FullAccess
- AmazonRekognitionFullAccess
- AmazonSNSFullAccess
- AWSLambdaBasicExecutionRole

Purpose: The role allows Lambda to interact with AWS services securely.



Step 4: Create Lambda Function

Lambda function was created using:

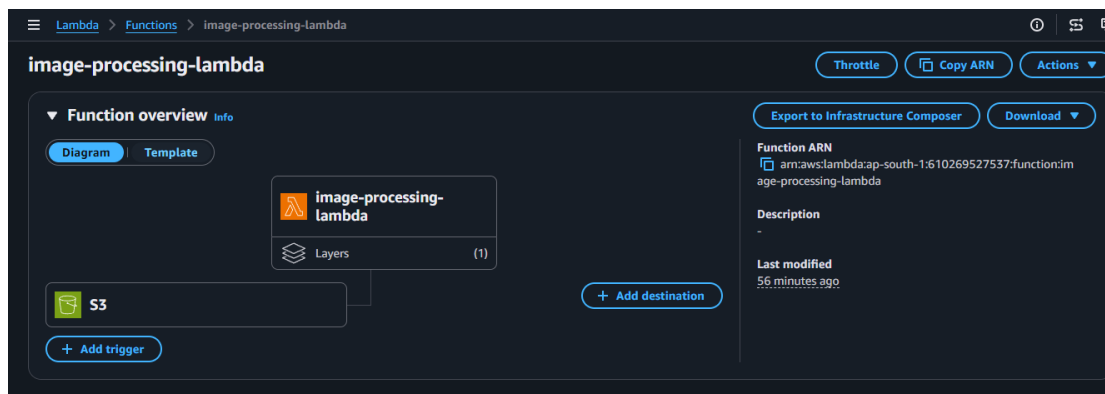
- Runtime: Python 3.12
- Architecture: x86_64

Trigger

S3 ObjectCreated event

Timeout

1 minute



Step 5: Add Pillow Layer

Pillow library was added using Lambda Layer.

Layer ARN Used

[arn:aws:lambda:ap-south-1:770693421928:layer:Klayers-p312-pillow:2](#)

Purpose: Pillow library is used for image resizing.

Layers (1) Info					
Last fetched 5/21/2026, 5:49:15 PM Refresh Edit					
Q Search by attributes or search by keyword					
Merge order	Name	Layer version	Compatible runtimes	Compatible architectures	Version ARN
1	Klayers-p312-pillow	2	python3.12	x86_64	arn:aws:lambda:ap-south-1:770693421928:layer:Klayers-p312-pillow:2

Step 6: Configure S3 Trigger

S3 trigger was attached to Lambda.

Event Type

- ObjectCreated

Purpose: Whenever a new image is uploaded, Lambda runs automatically.

Triggers (1) Info		Refresh	Fix errors	Edit	Delete	Add trigger
Q Search for triggers		< 1 >				
☐	Trigger					
☐	 S3: mudasir-input-bucket arn:aws:s3::mudasir-input-bucket Details					

Step 7: Lambda Workflow

The Lambda function performs the following tasks:

1. Receives uploaded image from S3
2. Resizes image using Pillow
3. Detects labels using Rekognition
4. Stores resized image in output bucket
5. Creates JSON file with labels
6. Sends SNS email notification

Final Lambda Code

```
import json
import boto3
import urllib.parse
from PIL import Image
from io import BytesIO

s3 = boto3.client("s3")
rekognition = boto3.client("rekognition")
sns = boto3.client("sns")

OUTPUT_BUCKET = "mudasir-output-bucket"
SNS_TOPIC_ARN = "arn:aws:sns:ap-south-1:610269527537:mudasir-topic"

def lambda_handler(event, context):
```

```

print("Event:", json.dumps(event))

bucket = event["Records"][0]["s3"]["bucket"]["name"]
key = urllib.parse.unquote_plus(event["Records"][0]["s3"]["object"]["key"])

# Read image from input S3 bucket
image_object = s3.get_object(Bucket=bucket, Key=key)
image_content = image_object["Body"].read()

# Resize image
image = Image.open(BytesIO(image_content)).convert("RGB")
resized_image = image.resize((300, 300))

buffer = BytesIO()
resized_image.save(buffer, format="JPEG")
buffer.seek(0)

resized_key = "resized-" + key

s3.put_object(
    Bucket=OUTPUT_BUCKET,
    Key=resized_key,
    Body=buffer,
    ContentType="image/jpeg"
)

# Detect labels using Rekognition
response = rekognition.detect_labels(
    Image={
        "S3Object": {
            "Bucket": bucket,
            "Name": key
        }
    },
    MaxLabels=10,
    MinConfidence=70
)

labels = [label["Name"] for label in response["Labels"]]

# Save labels as JSON
json_key = key.rsplit(".", 1)[0] + ".json"

result = {
    "original_file": key,
    "resized_file": resized_key,
    "labels": labels
}

s3.put_object(
    Bucket=OUTPUT_BUCKET,

```



```

        Key=json_key,
        Body=json.dumps(result),
        ContentType="application/json"
    )

    # Send SNS email
    message = f"""
Image processed successfully.

Original File: {key}
Resized File: {resized_key}
JSON File: {json_key}

Detected Labels:
{', '.join(labels)}
"""

    sns.publish(
        TopicArn=SNS_TOPIC_ARN,
        Subject="Image Processing Completed",
        Message=message
    )

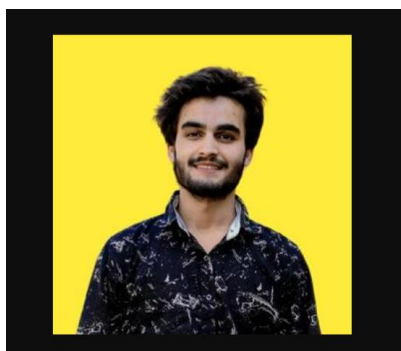
    print("Processing completed:", result)

    return {
        "statusCode": 200,
        "body": json.dumps(result)
    }

```

Output Files

Resized Image



☐	resized-172069802.jpg	jpg	May 21, 2026, 16:54:05 (UTC+05:30)	13.0 KB	Standard
--------------------------	---------------------------------------	-----	------------------------------------	---------	----------

JSON Output

☐	172069802.json	json	May 21, 2026, 16:54:05 (UTC+05:30)	175.0 B	Standard
--------------------------	--------------------------------	------	------------------------------------	---------	----------

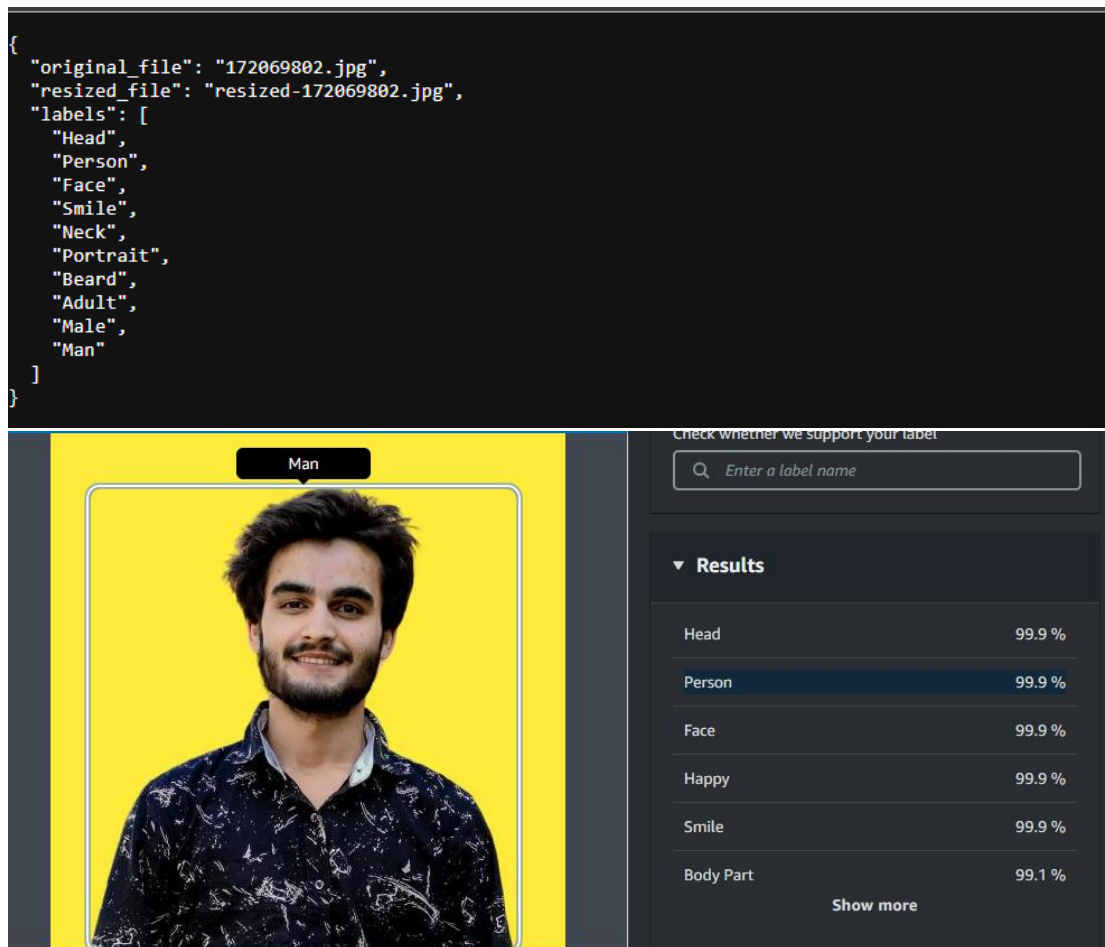
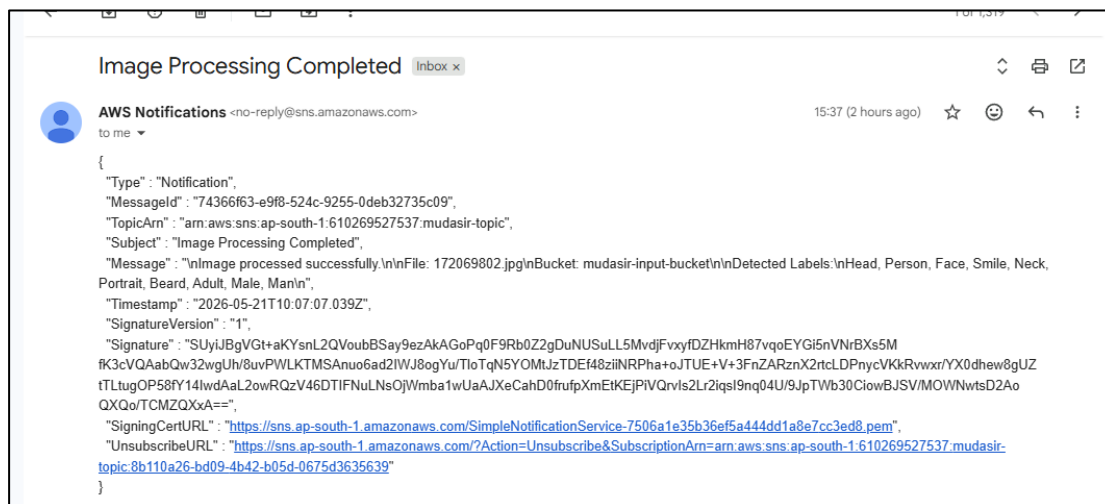


Image Processing Complete Mail:



CloudWatch Logs

CloudWatch logs were used for:

- Debugging Lambda execution
- Monitoring errors
- Viewing Rekognition labels
- Verifying image processing workflow



Troubleshooting

Issue 1: SNS Subscription Automatically Deactivated

Cause

SNS topic was created in different region.

Solution

- Used same AWS region for all services
- Recreated SNS topic in Mumbai region

Issue 2: Pillow Layer Compatibility Error

Cause

Lambda runtime was Python 3.14.

Solution

Created new Lambda using Python 3.12.

Real World Use Cases

- Face recognition systems
- Social media image analysis
- Smart surveillance systems
- Content moderation
- E-commerce product tagging
- AI photo organization systems

Conclusion

This project successfully demonstrates a real-world serverless AI pipeline using AWS cloud services. The application automatically processes uploaded images, resizes them, detects image labels using Amazon Rekognition, stores metadata in S3, and sends SNS notifications.

The architecture follows modern cloud-native and event-driven design principles and demonstrates practical implementation of AWS serverless services.

The project also provides hands-on experience with cloud automation, AI integration, Lambda event handling, IAM permissions, and scalable serverless architecture.